FORMAL METHODS REPORT

Lido
SRv3 Accounting Proof Closure

Executable evidence package for Lido's SRv3 economic accounting state machine. The selected P0 properties are checked over a focused Verity-style model with explicit source anchors and assumptions.

Version	Private Report v1.0
Date	June 2026
Commit	`PR-1811 / d088bbc2`
Reviewers	LFG Labs

Executable evidence for smart-contract state-machine review.

lfglabs.dev

Confidentiality and result status

This document is a **private executable evidence package** for the selected SRv3 accounting proof-closure lane. It connects the report claims to local model files, reference tests, proof target files, and reproducibility commands for PR #1811 at the commit named on the cover.

The report is intentionally narrow. It covers the economic state machine modeled for deposits, reserves, module balances, accepted reports, rewards, fees, and status gates. It does not claim full deployed-system equivalence, oracle truthfulness, cryptographic correctness, governance correctness, or exact proxy/storage refinement beyond the named assumptions.

Every selected target row is backed by a source-code cross-reference, trust-boundary entry, Verity/Lean theorem, and rebuild command. These artifacts are checked over the focused economic model; they are not full deployed-system refinement proofs.

> **Distribution**
>
> Private review channel for Lido protocol contributors and selected technical reviewers. Public release, forum excerpts, or website publication should keep the same bounded-check language unless stronger theorem-checker artifacts are added and independently rebuilt.

Contents

1 Executive summary

Question answered by this report

If PR #1811 changes how Lido accounts for deposit allocation, deposits, top-ups, reserves, module balances, and rewards, can the selected accounting rules be stated precisely enough that a machine can check them in a reproducible focused model? This report package gives an executable local answer for the fifteen selected P0 (highest-priority) properties over the focused economic model, under the named assumptions in Section 3.

In this repository, "executable" has a specific meaning: the model target is listed, all premises not discharged inside the model are registered by assumption ID, and `make test` plus `make prove` recheck the target set from a clean checkout.

The story in one page

The core safety question is simple enough to ask without Solidity or Lean: *when ETH and validator balances move through SRv3 accounting, can any value be spent from the wrong place, counted twice, rewarded from stale data, or sent through a disabled path?* The report answers that question by turning each informal concern into a small conservation law.

A useful mental model is three ledgers that must agree:

1. The **buffer ledger** says which ETH is depositable and which ETH is reserved for withdrawals.
2. The **router ledger** says how much ETH is pulled for deposits and how validator balances are summed across staking modules.
3. The **reward ledger** says which accepted balances may be used to compute rewards, fees, and recipients.

An "accepted report" means oracle-provided module-balance data that has passed the modeled SRv3 validation checks and is therefore allowed to update accounting state. The proof lane checks that PR #1811 preserves the three ledgers through those modeled transitions. It is not trying to prove every operational fact about deployment, cryptography, governance, gas, or the consensus layer.

Why this is more than testing

A test asks, "does this example work?" A Verity/Lean proof target asks, "does this rule follow from the modeled transition for all values covered by the theorem statement?" That is why the report spends so much effort naming the model and assumptions: they define exactly what the local command checks.

The executable claim this report package supports is:

> For the focused SRv3 economic state machine, the Verity/Lean model preserves deposit and top-up conservation, reserve separation, module-balance conservation, allocation-capacity bounds, report-before-reward consistency, reward/fee bounds, and flow-specific status gating under the named assumptions.

This is not a claim that every deployed-system behavior is proved. The exact boundary is part of the result: cryptographic witnesses, consensus truthfulness, governance configuration, and deployment refinement are named in Section 3.

How to read the proof

Read each row below as a three-step trace. First, the Verity/Lean model names the transition being studied. Second, the property states what must not go wrong in the focused model. Third, Lake checks the theorem file against the explicit assumptions.

The matrix answers fifteen plain questions: what ETH is spendable, how module capacity is bounded, what initial deposits and top-ups consume, what balances add up to, what report rewards use, how fees are bounded, and which module states block economic flow.

Proof matrix

All fifteen rows below have executable artifacts in `proofs/`, `LidoSRv3/`, and `verity/targets/`.

ID	PR #1811 surface	Executable property
SRV3-P1	Lido buffer allocation	Withdrawal-reserved ETH is excluded from the depositable resource, so deposit spending cannot borrow from withdrawal liquidity.
SRV3-P2	Router deposit path	Router-pulled ETH equals 32 ETH times actual deposits and is fully sunk into CL deposits after requiring enough depositable ETH and reducing modeled buffered ETH by that exact amount; allocated-deposit sums match actual deposits after an existing active module passes the nonzero-capacity and max-deposit guards. The selected module's last-deposit field records that exact pulled amount through an exact module-array update while preserving module-array length and module-balance sums, with no modeled residual held by the router, no deposit- or withdrawal-reserve field mutation, and no router report-state mutation. Zero-deposit returns still record last-deposit accounting but do not move modeled buffer or beacon-sink value.
SRV3-P3	Module balances	Router-wide validator balance is exactly the sum of accepted per-module balances, the post-state module array is exactly the accepted report update-loop result, the successful report loop preserves module-array length and ETH-side state, and the exact accepted report is stored for later reward reads.
SRV3-P4	Oracle report ordering	Module fee-distribution weights come from the current accepted module-balance state, not from stale or partially applied reports.
SRV3-P5	Rewards and fees	Reward distribution returns empty rows and an empty module-id projection when total balance is zero; otherwise rows use accepted nonzero module balances, skip zero-balance modules, keep recipients aligned, bound paid module fees, and pay stopped modules zero module-side fee, with total fee equal to the row-wise module plus treasury fee sum.
SRV3-P6	Status gates	Non-active or deposit-paused modules receive no modeled deposit allocation; stopped modules receive no module-side reward or fee allocation.
SRV3-P7	Exited-count updates	Successful exited-count rows name existing modules, cannot decrease the router-stored count, cannot exceed deposited validators, return the exact sequential loop result, and preserve module-array length, module-balance sums, and non-module router state.
SRV3-P8	Router top-up path	Top-up allocations are same-length, Gwei-aligned, within per-key limits and the rounded module target, which keeps returned sums inside the module allocation budget. Nonzero sums require enough depositable ETH before the modeled Lido pull, reduce modeled buffered ETH by that exact sum, and are fully sunk; zero sums return the unchanged state. Both branches leave no modeled router residual or reserve-field mutation, and do not mutate router module or report accounting.
SRV3-P9	Deposit allocation capacity	SRv3 allocation-capacity rows preserve router module length and module-id order, and derived capacity values preserve router module length; active capacity is bounded by target share and available capacity, while inactive modules keep current allocation.
SRV3-P10	Reward-minted reporting	Reward-minted report arrays have equal length and return exactly one zipped row per module id; zero-share rows are skipped, and every nonzero-share row names an existing module before callback.
SRV3-P11	All-module fee updates	Fee-update arrays match module count, keep every module/treasury fee sum within total basis points, require a consistent fee sum across modules, set fee projections exactly to the supplied arrays, and preserve module-list length, validator-balance sums, and non-module router state.
SRV3-P12	Single-module config	Single-module updates require an existing module, valid share parameters, bounded and consistent fee sums, uint64 deposit-parameter bounds, exact selected-module field

Review posture

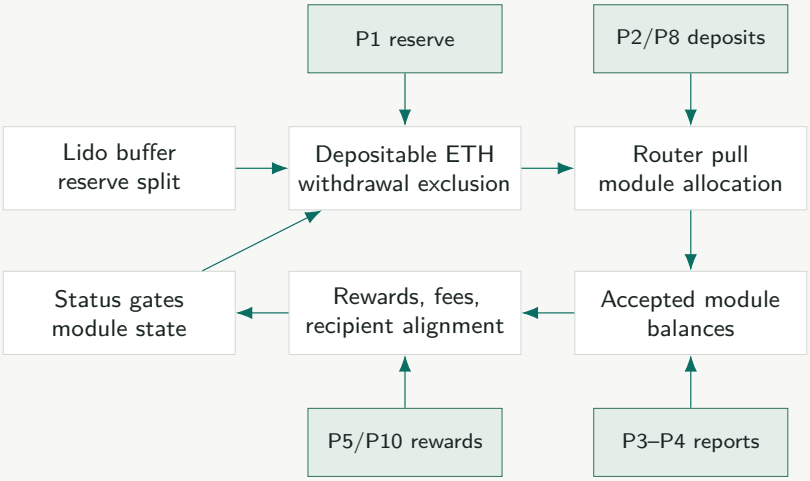
This report package is written for two readers at once. A protocol reviewer can follow the ledgers and the trust boundary without reading code. A formal-methods reviewer can check whether the target IDs, assumptions, and rebuild gates support the advertised local executable claim.

2 What the proof closes

A local proof target is closed in this package when the executable model check passes, the model and source anchors are listed, and the remaining premises are named as assumptions. This section shows the fifteen accounting claims in three forms: a system map, a plain-language story, and artifact rows for the evidence package.

Modeled architecture

The proof model follows the accounting path where PR #1811 concentrates fund-safety risk. In plain terms, the model follows ETH as it moves from the Lido buffer, through the router, into staking-module deposits, and then into later balance reports and reward calculations. Each proof asks whether that flow preserves the accounting rule it is supposed to preserve. External contracts are summarized only at the interfaces used by those transitions.



From intuition to target

Read the fifteen properties as one accounting story. P1 protects ETH reserved for withdrawals. P2 checks that initial deposits pull exactly the ETH they should. P3 checks that the router's total balance equals the module balances it accepted. P4 makes module fee weights depend on the accepted report, not an old or partial one. P5 bounds reward and fee movement. P6 blocks non-active or paused modules from deposit allocation and stopped modules from module-side reward or fee allocation. P7 checks exited-count updates. P8 checks that top-up allocation arrays are bounded and fully sunk. P9 checks SRv3's per-module allocation-capacity rows before the external allocation strategy runs. P10 checks the SRLib reward-minted reporting loop before module callbacks are attempted. P11 checks the all-module fee-update arrays and module-config update loop. P12 checks the single-module config-update guards that feed deposit allocation and reward-fee configuration. P13 checks the governance-controlled module-status update path that feeds the status-gating rules without changing validator-balance accounting. P14 checks the module-creation path that extends the router module array with a fresh, zero-accounting module. P15 checks the all-module share-update loop that changes target-share and priority-exit thresholds without changing module accounting balances.

Each invariant below has two layers. The first sentence says why the property matters in ordinary accounting language. The formula then states the same idea in a form that can be checked over all modeled inputs. You can read the formulas like receipts: they name

the money bucket before and after a move, then check that the totals still match. The symbols below only stand for modules, balances, reserves, and reports already described in the ledger story.

Let M be the finite set of staking modules in router order. For a module $m \in M$, let bal_m be its accepted validator balance in gwei, status_m its module status, and $\mathsf{moduleRecipient}_m$ its module reward recipient in the modeled reward step. Let B_R be the router validator balance, T the buffered ETH, R_D the stored deposit reserve, and R_W the withdrawal-reserved buffer. The checked theorems are in `LidoSRv3/SpecProofs.lean`. Each accepted report must still cover the current router module set in router order, with unique module IDs, matching array lengths, and the freshness and range premises named in the target file.

P1: reserve separation.

$$d = \min(T, R_D) \qquad w = \min(T - d, R_W) \qquad u = T - d - w$$

$$\mathsf{depositable}(T) = d + u$$

$$\forall a.\quad \mathsf{spendDeposit}(a) \Rightarrow a \le \mathsf{depositable}(T)$$

$$\mathsf{spendDeposit}(a) \Rightarrow \mathsf{usesWithdrawalReserve}(a, T, R_D, R_W) = 0$$

Plain reading: some ETH is saved for withdrawals, and deposits are not allowed to spend that saved amount. Deposit spend is bounded by the pre-spend deposit reserve plus unreserved allocation and does not consume the pre-spend effective withdrawal-reserved bucket. The executable target deliberately avoids a raw post-state equality for R_W; any post-state reserve fact belongs to the model's explicit reserve-recomputation rule.

P2: exact router pull.

$$\mathsf{pulledWei} = 32\,\mathsf{ETH} \cdot \mathsf{actualDeposits} \qquad \mathsf{routerEthAfter} = \mathsf{routerEthBefore}$$

$$\mathsf{actualDeposits} = \sum_{m \in M} \mathsf{allocatedDeposits}_m$$

$$\mathsf{actualDeposits} > 0 \Rightarrow \mathsf{pulledWei} \le \mathsf{depositableEtherBefore}$$

$$\mathsf{actualDeposits} > 0 \Rightarrow \mathsf{bufferedEtherAfter} = \mathsf{bufferedEtherBefore} - \mathsf{pulledWei}$$

$$\mathsf{actualDeposits} = 0 \Rightarrow \mathsf{bufferedEtherAfter} = \mathsf{bufferedEtherBefore} \quad \wedge \quad \mathsf{beaconSinkAfter} = \mathsf{beaconSinkBefore}$$

$$\mathsf{withdrawalReserveAfter} = \mathsf{withdrawalReserveBefore} \qquad \mathsf{depositReserveAfter} = \mathsf{depositReserveBefore} - \mathsf{pulledWei}$$

$$\mathsf{module.status} = \mathsf{active} \quad \wedge \quad \mathsf{maxDepositsCount} \neq 0 \quad \wedge \quad \mathsf{actualDeposits} \le \mathsf{maxDepositsCount}$$

$$M' = \mathsf{recordModuleLastDeposit}(\mathsf{stakingModuleId}, \mathsf{pulledWei}, M)$$

$$\mathsf{selectedModule.lastDepositWeiAfter} = \mathsf{depositPullWei}(\mathsf{actualDeposits})$$

$$|M'| = |M|$$

$$\sum_{m \in M'} \mathsf{bal}_m = \sum_{m \in M} \mathsf{bal}_m$$

$$\mathsf{routerBalanceGweiAfter} = \mathsf{routerBalanceGweiBefore} \quad \wedge \quad \mathsf{lastAcceptedReportAfter} = \mathsf{lastAcceptedReportBefore}$$

Plain reading: across the modeled atomic initial-deposit transition, the router pulls exactly one 32 ETH unit per validator only when that value is available as depositable ETH, subtracts exactly that amount from modeled buffered ETH, and immediately forwards it to the beacon deposit sink without consuming the withdrawal-reserved bucket. The stored deposit-reserve bucket is reduced by the same spend amount, saturating at zero, matching

the Lido buffer spend rule. The zero-key return path records the selected module's last-deposit field but does not move modeled value out of the buffer or into the beacon sink. For both branches, the whole module array is exactly the recordModuleLastDeposit transformation of the pre-state array. The successful transition also exposes the Solidity guard facts: the selected module exists, is active, has nonzero computed capacity, and the returned deposit count is within that capacity. It also checks the side effect that Solidity performs before the zero-key return: the selected module's last-deposit field records the exact pulled value, including zero, while preserving module-array length, the router module-balance sum, and the stored router report state. Top-ups are a separate Gwei-aligned allocation target covered by P8, not instances of this 32 ETH exact-pull statement.

P3: module-balance conservation.

$$\mathsf{acceptedReport}(M, \vec{b}) \Rightarrow B'_R = \sum_{m \in M} \mathsf{bal}'_m = \sum_{i=0}^{|M|-1} \vec{b}_i$$

$$\mathsf{successfulReportTransition}(r) \Rightarrow \mathsf{moduleBalances}' = \mathsf{reportBalances}(r)$$

$$\mathsf{successfulReportTransition}(r) \Rightarrow M' = \mathsf{applyReportToModules}(M, r)$$

$$\mathsf{successfulReportTransition}(r) \Rightarrow \mathsf{lastAcceptedReport}' = r$$

$$\mathsf{successfulReportTransition}(r) \Rightarrow |M'| = |M|$$

$$\mathsf{successfulReportTransition}(r) \Rightarrow \mathsf{ETHState}' = \mathsf{ETHState}$$

Plain reading: the router total must equal the sum of the module rows after a successful module-balance report transition. The modeled transition preserves the Solidity length, router-order module-ID, and Gwei range checks before applying the update loop, writes the submitted balance list into the module balance fields exactly, proves the whole post-state module array is the accepted report update-loop result, preserves the module-array length and ETH-side state after that loop, then records the exact accepted report for the reward-distribution read path. The proof is over arbitrary finite module arrays; unique module IDs remain a named well-formedness assumption rather than a hidden bound on module count.

P4: report-before-reward consistency.

$$\mathsf{rewardStep}(s, s') \Rightarrow \exists r.\, \mathsf{acceptedAt}(r, s) \wedge \mathsf{rewardBase}(s) = \mathsf{balances}(r)$$

Plain reading: accepted module balances determine the module fee-distribution weights used by the modeled reward step. Total reward magnitude and protocol fee inputs are treated as accepted report or vault-context inputs subject to the listed range and freshness assumptions. In simpler terms, rewards should be split using the report the system just accepted, not a previous snapshot and not a half-updated table.

P5: reward-distribution loop and fee alignment.

$$\mathsf{share}_m = \lfloor \mathsf{bal}_m \cdot \mathsf{precision} / \mathsf{totalBalance} \rfloor$$

$$\mathsf{moduleFee}_m = \lfloor \mathsf{share}_m \cdot \mathsf{moduleFeeBps}_m / 10{,}000 \rfloor$$

$$\mathsf{treasuryFee}_m = \lfloor \mathsf{share}_m \cdot \mathsf{treasuryFeeBps}_m / 10{,}000 \rfloor$$

$$\mathsf{bal}_m = 0 \Rightarrow m \notin \mathsf{rewardRows} \qquad \mathsf{row} \in \mathsf{rewardRows} \Rightarrow \mathsf{row.balance} \neq 0$$

$$\mathsf{paidModuleFee}_m \leq \mathsf{moduleFee}_m$$

$$\text{totalBalance} = 0 \Rightarrow \text{rewardRows} = [\,] \wedge \text{map}(\text{row.moduleId}, \text{rewardRows}) = [\,]$$

$$\text{status}_m = \text{stopped} \Rightarrow \text{paidModuleFee}_m = 0$$

$$\text{totalFee} = \sum_{\text{row} \in \text{rewardRows}} (\text{row.moduleFee} + \text{row.treasuryFee})$$

Plain reading: the modeled `getStakingRewardsDistribution` loop returns no rows when the router total validator balance is zero, and the returned module-id projection is empty in that branch. Otherwise it emits rows only for modules with nonzero accepted validator balance, keeps each row paired with the module's recipient, bounds paid module-side fees by the computed module fee, and pays stopped modules zero module-side fee. The total fee modeled for the final assertion is the sum of every emitted row's module and treasury fee components.

P6: status gating.

$$\text{status}_m \neq \text{active} \vee \text{depositsPaused}_m \Rightarrow \text{allocatedDeposits}_m = 0$$

$$\text{status}_m = \text{stopped} \Rightarrow \text{moduleReward}_m = 0$$

Plain reading: a stopped or paused module is not just marked differently in storage; that status changes the allowed flow. Deposit-paused or non-active modules receive zero modeled deposit allocation. Stopped modules receive zero module-side fee or reward allocation. Any residual protocol-fee treatment is stated separately from module recipient payment.

P7: exited-count update correctness.

$$\text{successfulExitedUpdate}(m, e') \Rightarrow \text{existsModule}(m) \wedge e_m \leq e' \wedge e' \leq \text{depositedValidators}_m$$

$$\text{successfulExitedUpdate}(\text{rows}) \Rightarrow |M'| = |M|$$

$$\text{successfulExitedUpdate}(\text{rows}) \Rightarrow (M', \text{newlyExited}) = \text{sequentialExitedLoop}(M, \text{rows})$$

$$\text{successfulExitedUpdate}(\text{rows}, S, S') \Rightarrow$$
$$\text{nonModuleRouterState}' = \text{nonModuleRouterState}$$

Plain reading: an accepted exited-count row cannot invent a module, cannot move the router-stored exited count backward, and cannot report more exited validators than the staking module summary says were deposited. The model preserves Solidity's sequential loop behavior over arbitrary finite row arrays, including duplicate module IDs, keeps the module array length unchanged, and preserves the router module-balance sum and non-module router state. It also proves that successful transitions return exactly the sequential loop result and that the empty-row path returns the unchanged state with zero newly exited validators.

P8: top-up allocation conservation.

$$\text{successfulTopUp}(m, \vec{a}, \vec{\ell}) \Rightarrow \text{active}(m) \wedge \text{supportsTopUp}(m)$$

$$\text{keyCount} > 0 \wedge |\vec{operatorIds}| = |\vec{\ell}| = |\vec{pubkeys}| = \text{keyCount}$$

$$|\vec{a}| = |\vec{\ell}| \wedge \forall i.\ \vec{a}_i \equiv 0 \pmod{1\,\text{gwei}} \wedge \forall i.\ \vec{a}_i \leq \vec{\ell}_i$$

$$\sum_i \vec{a}_i \leq \text{roundDownToGwei}(\text{moduleAllocationWei}) \leq \text{moduleAllocationWei}$$

$$\sum_i \vec{a}_i > 0 \Rightarrow \sum_i \vec{a}_i \leq \text{depositableEtherBefore}$$

$$\sum_i \vec{a}_i > 0 \Rightarrow \text{bufferedEtherAfter} = \text{bufferedEtherBefore} - \sum_i \vec{a}_i$$

$$\sum_i \vec{a}_i > 0 \Rightarrow \text{beaconTopUpSink}' = \text{beaconTopUpSink} + \sum_i \vec{a}_i$$

$$\sum_i \vec{a}_i = 0 \Rightarrow S' = S$$

$$\sum_i \vec{a}_i > 0 \Rightarrow \text{routerEthAfter} = \text{routerEthBefore}$$

$$M' = M$$

$$\text{routerBalanceGwei}' = \text{routerBalanceGwei} \qquad \text{lastAcceptedReport}' = \text{lastAcceptedReport}$$

$$\text{withdrawalReserveAfter} = \text{withdrawalReserveBefore} \qquad \text{depositReserveAfter} = \text{depositReserveBefore} - \sum_i \text{allocati}$$

Plain reading: successful modeled top-ups preserve the Solidity array checks around `allocateDeposits`, including nonempty input shape, same returned allocation length, Gwei alignment, per-key limits, and the rounded module target, which also proves the returned sum stays inside the module allocation budget. If the allocation sum is positive, the router pulls exactly that amount from Lido only when it is available as depositable ETH, then forwards it to the top-up deposit sink after subtracting the same amount from modeled buffered ETH and the stored deposit reserve, leaving no modeled router ETH residual and no withdrawal-reserve consumption, and without mutating router module or report accounting. If the checked allocation sum is zero, the modeled transition returns the unchanged state after validation.

P9: allocation-capacity rows.

$$\text{row} \in \text{modulesAllocationAndCapacity}(M) \Rightarrow \exists m \in M.\ \text{row} = \text{capacityRow}(m)$$

$$|\text{modulesAllocationAndCapacity}(M)| = |M|$$

$$|\text{allocatedCapacityValues}(\text{modulesAllocationAndCapacity}(M))| = |M|$$

$$\text{map}(\text{row.moduleId}, \text{modulesAllocationAndCapacity}(M)) = \text{map}(\text{module.id}, M)$$

$$\text{active}(m) \Rightarrow \text{capacity}_m \leq \text{targetValidators}_m \wedge \text{capacity}_m \leq \text{availableCapacity}_m$$

$$\neg\text{active}(m) \Rightarrow \text{capacity}_m = \text{currentAllocation}_m$$

Plain reading: the modeled SRv3 allocation-capacity loop emits one row per router module, preserves the module-id order in those rows, and keeps the capacity-value array length aligned with the module array. Active module capacity is bounded by both the stake-share target and the available module capacity computed for initial deposit or top-up mode. Inactive modules do not gain capacity; their capacity stays equal to their current allocation. The external MinFirst strategy that turns these arrays into final deltas remains a named assumption.

P10: reward-minted reporting guards.

$$\text{successfulRewardMintedReport}(\vec{id}, \vec{s}, \vec{r}) \Rightarrow |\vec{id}| = |\vec{s}|$$

$$\text{successfulRewardMintedReport}(\vec{id}, \vec{s}, \vec{r}) \Rightarrow \vec{r} = \text{zipRows}(\vec{id}, \vec{s})$$

$$\text{successfulRewardMintedReport}(\vec{id}, \vec{s}, \vec{r}) \Rightarrow |\vec{r}| = |\vec{id}|$$

$$\text{successfulRewardMintedReport}(\vec{id}, \vec{s}, \vec{r}) \Rightarrow$$
$$\text{mapModuleIds}(\vec{r}) = \vec{id} \wedge \text{mapTotalShares}(\vec{r}) = \vec{s}$$

$$row \in \vec{r} \wedge row.\text{totalShares} \neq 0 \Rightarrow \text{existsModule}(row.\text{moduleId})$$

$$row.\text{totalShares} = 0 \Rightarrow \text{skipBeforeModuleCheck}(row)$$

Plain reading: the modeled `_reportRewardsMinted` loop first checks equal calldata-array lengths, constructs exactly one report row per zipped module-id/share pair, preserves the module-id and share arrays in the returned row projections, skips zero-share rows, and requires an existing module id before any nonzero-share callback can be attempted. Callback effects, low-level revert bytes, and event behavior remain named assumptions.

P11: all-module fee-update consistency.

$$\text{successfulFeeUpdate}(\vec{f}, \vec{t}, M, M') \Rightarrow |\vec{f}| = |M| \wedge |\vec{t}| = |M|$$

$$\text{successfulFeeUpdate}(\vec{f}, \vec{t}, M, M') \Rightarrow \forall i.\ \vec{f}_i + \vec{t}_i \leq 10{,}000$$

$$\text{successfulFeeUpdate}(\vec{f}, \vec{t}, M, M') \Rightarrow |M'| = |M|$$

$$\text{map}(\text{moduleFeeBps}, M') = \vec{f} \qquad \text{map}(\text{treasuryFeeBps}, M') = \vec{t}$$

$$\text{successfulFeeUpdate}(\vec{f}, \vec{t}, M, M') \Rightarrow \sum_{m \in M'} \text{balance}_m = \sum_{m \in M} \text{balance}_m$$

$$\text{successfulFeeUpdate}(\vec{f}, \vec{t}, S, S') \Rightarrow \text{nonModuleRouterState}' = \text{nonModuleRouterState}$$

Plain reading: the modeled `_updateAllModuleFees` loop checks that both fee arrays match router module count, accepts the empty-module case, requires every row's module-plus-treasury fee sum to fit within total basis points, requires all row sums to match the first row's sum, and updates module fee and treasury-fee projections exactly to the supplied arrays without changing the module-list length or router-stored validator-balance sum. It also preserves ETH accounting, router report balance, and the last accepted report.

P12: single-module config-update consistency.

$$\text{successfulModuleParamUpdate}(m, p, M, M') \Rightarrow \text{existsModule}(m)$$

$$\text{successfulModuleParamUpdate}(m, p, M, M') \Rightarrow$$
$$\text{stakeShareLimit}_p \leq \text{priorityExitShareThreshold}_p \leq 10{,}000$$

$$\text{successfulModuleParamUpdate}(m, p, M, M') \Rightarrow$$
$$\text{moduleFee}_p + \text{treasuryFee}_p \leq 10{,}000 \wedge |M'| = |M|$$

$$\text{successfulModuleParamUpdate}(m, p, M, M') \Rightarrow \sum_{m' \in M'} \text{balance}_{m'} = \sum_{m' \in M} \text{balance}_{m'}$$

$$\text{successfulModuleParamUpdate}(m, p, M, M') \Rightarrow$$
$$\text{selectedModuleParams}(m, M') = \text{requestedParams}(p)$$

$$\text{successfulModuleParamUpdate}(m, p, S, S') \Rightarrow$$
$$\text{nonModuleRouterState}' = \text{nonModuleRouterState}$$

Plain reading: the modeled `_updateModuleParams` path checks module existence, stake-share and priority-exit threshold ordering, bounded module-plus-treasury fees, fee-sum consistency against every other module, nonzero minimum deposit block distance, and

uint64 bounds for deposit parameters before updating the selected module without changing module-list length or router-stored validator-balance sums. The post-state lookup for the selected module records exactly the requested share, fee, max-deposit, and min-distance fields. It also preserves ETH accounting, router report balance, and the last accepted report.

P13: module-status update consistency.

$$\text{successfulStatusUpdate}(m, \sigma, M, M') \Rightarrow \text{existsModule}(m) \wedge \text{oldStatus}_m \neq \sigma$$

$$\text{successfulStatusUpdate}(m, \sigma, M, M') \Rightarrow |M'| = |M|$$

$$\text{successfulStatusUpdate}(m, \sigma, M, M') \Rightarrow \text{statusOf}(m, M') = \sigma$$

$$\text{successfulStatusUpdate}(m, \sigma, M, M') \Rightarrow \sum_{m' \in M'} \text{validatorsBalanceGwei}_{m'} = \sum_{m \in M} \text{validatorsBalanceGwei}_m$$

$$\text{successfulStatusUpdate}(m, \sigma, S, S') \Rightarrow \text{nonModuleRouterState}' = \text{nonModuleRouterState}$$

Plain reading: the modeled `_setModuleStatus` path checks that the selected module exists, rejects unchanged public status updates, updates router-stored module status so the selected module records the requested status, and preserves both module-list length and validator-balance accounting. It also preserves ETH accounting, router report balance, and the last accepted report.

P14: module-addition consistency.

$$\text{successfulAddModule}(m, p, M, M') \Rightarrow \neg\text{existsModule}(m, M)$$

$$\text{successfulAddModule}(m, p, M, M') \Rightarrow \text{shareAndFeeConfigValid}(p, M) \wedge |M'| = |M| + 1$$

$$\text{successfulAddModule}(m, p, M, M') \Rightarrow M' = M +\!\!+ [\text{newModuleFromConfig}(p)]$$

$$\text{successfulAddModule}(m, p, M, M') \Rightarrow \sum_{m' \in M'} \text{validatorsBalanceGwei}_{m'} = \sum_{m \in M} \text{validatorsBalanceGwei}_m$$

$$\text{successfulAddModule}(m, p, S, S') \Rightarrow \text{nonModuleRouterState}' = \text{nonModuleRouterState}$$

Plain reading: the modeled `_addModule` path requires a fresh module id, validates the new module's share, fee, fee-sum consistency, deposit distance, and uint64-bounded deposit parameters, appends exactly the module built from that accepted config, starts it active with zero initial accounting fields, and leaves the validator-balance sum unchanged. It also preserves ETH accounting, router report balance, and the last accepted report.

P15: all-module share-update consistency.

$$\text{successfulShareUpdate}(\vec{s}, \vec{p}, M, M') \Rightarrow |\vec{s}| = |M| \wedge |\vec{p}| = |M|$$

$$\text{successfulShareUpdate}(\vec{s}, \vec{p}, M, M') \Rightarrow \forall i.\ \vec{s}_i \leq \vec{p}_i \leq 10{,}000$$

$$\text{successfulShareUpdate}(\vec{s}, \vec{p}, M, M') \Rightarrow \text{map}(\text{stakeShareLimitBps}, M') = \vec{s} \wedge \text{map}(\text{priorityExitShareThresholdBps}, M') = \vec{p}$$

$$\text{successfulShareUpdate}(\vec{s}, \vec{p}, M, M') \Rightarrow |M'| = |M| \wedge \sum_{m' \in M'} \text{validatorsBalanceGwei}_{m'} = \sum_{m \in M} \text{validatorsBalanceGwei}_{m}$$

$$\text{successfulShareUpdate}(\vec{s}, \vec{p}, S, S') \Rightarrow \text{nonModuleRouterState}' = \text{nonModuleRouterState}$$

Plain reading: the modeled `_updateModuleShares` loop checks that both share arrays match router module count, validates each row's stake-share and priority-exit threshold ordering, updates share-field projections exactly to the supplied arrays in router order, and preserves both module-list length and validator-balance accounting. It also preserves ETH accounting, router report balance, and the last accepted report.

Report package traceability register

The next table is for auditability. The artifact path is the local file a reviewer can inspect or run. The assumption IDs say which facts the check is allowed to borrow from outside the model. The report does not ask the reader to trust prose alone; every claim has this chain.

ID	Executable artifact	Assumptions	Claim
SRV3-P1	`proofs/reserve-separation.md`	A-EXT-01; A-ARITH-05	Deposit spend cannot consume withdrawal-reserved ETH.
SRV3-P2	`proofs/deposit-conservation.md`	A-DEP-02; A-LIDO-06; A-MOD-07; A-ID-04; A-ARITH-05	Pulled wei equals 32 ETH times actual deposits; allocated-deposit sums, active-module and max-deposit guards, depositable-ETH preconditions, exact nonzero buffered-ETH decrease, zero-key external-value preservation, and exact module-array last-deposit recording are preserved while deposit reserves, module-balance sums, and report state stay unchanged.

ID	Executable artifact	Assumptions	Claim
SRV3-P3	`proofs/module-balance-conservation.md`	A-ORC-03; A-ID-04; A-ARITH-05	Successful report transitions check full length, router order, Gwei range, and write the exact accepted-report update-loop result while preserving module-array length and router balance as the module sum; ETH-side state remains unchanged.
SRV3-P4	`proofs/report-before-reward-consistency.md`	A-ORC-03; A-ID-04	Module fee-distribution weights read the latest accepted module-balance state.
SRV3-P5	`proofs/reward-correctness.md`	A-ORC-03; A-ARITH-05; A-REWARD-09	Reward distribution returns no rows and an empty module-id projection at zero total balance; otherwise rows use accepted balances, skip zero-balance modules, keep recipients aligned, and zero stopped-module paid fees; total fee is the row-wise module plus treasury fee sum.
SRV3-P6	`proofs/status-gating.md`	A-EXT-01; A-ID-04	Non-active or deposit-paused modules receive no modeled deposit allocation; stopped modules receive no module-side reward or fee allocation.

ID	Executable artifact	Assumptions	Claim
SRV3-P7	`proofs/exited-count-correctness.md`	A-MOD-08; A-ORC-03; A-ARITH-05	Successful exited-count rows name existing modules, cannot decrease, and cannot exceed deposited validators; successful loops return exactly the sequential loop result while preserving module-array length, the router module-balance sum, and non-module router state.
SRV3-P8	`proofs/topup-conservation.md`	A-LIDO-06; A-DEP-02; A-MOD-10; A-ARITH-05	Successful top-up transitions preserve allocation shape, keep returned sums inside the module allocation budget, require depositable ETH for nonzero pulls, decrease buffered ETH exactly by nonzero allocation sums, fully sink nonzero allocation sums, leave module and report accounting unchanged, preserve reserve fields, and leave zero-sum transitions unchanged.
SRV3-P9	`proofs/allocation-capacity.md`	A-ID-04; A-MOD-08; A-MOD-11; A-ALLOC-12; A-ARITH-05	Allocation-capacity rows preserve module length and module-id order, derived capacity values stay module-length aligned, and active/inactive capacity bounds hold before external MinFirst allocation.

ID	Executable artifact	Assumptions	Claim
SRV3-P10	`proofs/reward-minted-reporting.md`	A-EXT-01; A-MOD-13	Reward-minted report arrays are equal length and return one zipped row per module id with exact input-array projections; zero-share rows are skipped, and nonzero-share rows name existing modules before callback.
SRV3-P11	`proofs/fee-update-consistency.md`	A-GOV-14; A-ARITH-05	All-module fee updates require matching array lengths, consistent bounded fee sums, set fee projections exactly to the supplied arrays, and preserve module-list length and validator-balance sums after updating module configs while leaving non-module router state unchanged.
SRV3-P12	`proofs/module-param-update-consistency.md`	A-GOV-14; A-ID-04; A-ARITH-05	Single-module config updates require an existing module, valid share and deposit parameters, bounded consistent fee sums, record the requested fields for the selected module, and preserve module-list length, validator-balance sums, and non-module router state.
SRV3-P13	`proofs/module-status-update.md`	A-GOV-14; A-ID-04	Module-status updates require an existing module and changed status, record the requested status, preserve module-list length, and leave router-stored module validator-balance sums and non-module router state unchanged.

ID	Executable artifact	Assumptions	Claim
SRV3-P14	`proofs/module-addition-consistency.md`	A-GOV-14; A-ID-04; A-ARITH-05	Module addition requires a fresh id and valid config, appends exactly the accepted new module, starts it with zero accounting, and leaves router-stored validator-balance sums and non-module router state unchanged.
SRV3-P15	`proofs/module-share-update-consistency.md`	A-GOV-14; A-ARITH-05	All-module share updates require matching array lengths and valid share rows, set share projections exactly to the supplied arrays, then preserve module-list length, router-stored validator-balance sums, and non-module router state.

Evidence architecture

Layer	Artifact	Role
Solidity-facing summaries	`tests/solidity-reference/`	Preserve the relevant array, loop, reserve, and call structure while anchoring the model against copied PR #1811 test intent.
Economic model	`LidoSRv3/Model.lean`	Defines modules, reserves, Wei/Gwei arithmetic, status flags, accepted reports, rewards, and fee state.
Proof obligations	`LidoSRv3/SpecProofs.lean`	States each target over the focused transition model, with uniqueness, ordering, alignment, and freshness premises named in target files.
Trust-boundary report	`verity/targets/trust-boundary.json`	Machine-readable list of assumptions, target IDs, source links, retained Certora assumptions, and out-of-model claims.
Rebuild gate	`make test; make prove`	Checks source fixtures, runs the Verity/Lean theorem target, and emits `proofs/logs/proof-report.json`.

3 Trust boundary

A formal proof is not a proof of "everything the deployed system can ever do." It is a proof about a specific model. In plain terms, this model checks the accounting moves inside the selected SRv3 state machine; it does not prove that every outside system feeding that state machine is honest, available, or cryptographically sound. The boundary below is therefore part of the result, not a footnote.

What is inside the model

Inside	Why it is modeled
Reserve and buffer arithmetic	This is where depositable ETH must be separated from withdrawal-reserved ETH.
Router deposit and top-up accounting	This is where pulled ETH must match actual 32 ETH validator deposits or accepted top-up allocation sums.
Deposit allocation capacity	This is where SRv3 builds current-allocation and capacity arrays before calling the external allocation strategy.
Module-balance reports	This is where per-module balances become the router-wide accepted balance.
Rewards, fees, and recipients	This is where amount bounds and recipient alignment matter.
Module lifecycle and status gates	This is where module creation/config/status updates feed the finite router array, and where non-active or paused modules stop receiving modeled deposit allocation while stopped modules stop receiving module-side rewards or fees.

Named assumptions

An assumption is not a hidden conclusion. It is a fact this proof is allowed to start from. The point of naming assumptions is to show exactly which parts still need ordinary audit, deployment review, oracle review, or cryptographic review.

ID	Assumption	Meaning
A-EXT-01	External contract summaries	Lido buffer withdrawals, staking module callbacks, fee recipients, and beacon deposit sinks conform to the specified interfaces and mutate no unlisted SRv3 accounting state.

ID	Assumption	Meaning
A-GOV-14	Governance and config-call authorization	Governance role authorization, calldata authorship, and event semantics for module-management updates remain outside this lane. P11–P15 check the SRv3 fee, share, config, status, and add-module validation/update loops once invoked; module contract registration and address/interface validation are not proved.
A-DEP-02	Beacon deposit sink	A modeled beacon deposit or top-up removes exactly the intended value from the router-side resource. Deposit-root, BLS, dummy-signature behavior, minimum top-up amount checks, and SSZ correctness are not proved.
A-ORC-03	Accepted oracle reports	Off-chain oracle truthfulness remains assumed. The modeled SRv3 transition still checks full module-array length, router-order module IDs, and the SRv3 Gwei amount range before applying module balances.
A-ID-04	Module identity and ordering well-formedness	Module IDs are unique, report arrays align with those IDs where required, and sortedness obligations are enforced for ordered paths.
A-ARITH-05	Unit-conversion domain	Wei/Gwei and basis-point arithmetic is interpreted over the bounded unsigned integer domains used by the modeled transitions.
A-REWARD-09	Reward precision and cast bounds	Modeled reward fee arithmetic is assumed to stay within the Solidity `uint96` and high-precision domains, including the final `totalFee <= FEE_PRECISION_POINTS` assertion.
A-LIDO-06	Lido buffer withdrawal interface	A modeled `withdrawDepositableEther` call makes exactly the requested depositable amount available to the router and does not consume withdrawal-reserved ETH.
A-MOD-07	Staking module deposit-data interface	A modeled staking module returns the specified number of deposit pubkeys and does not mutate router accounting except through the explicit SRv3 transition. Pubkey bytes, signatures, and module internals are outside this lane unless a P0 property names them.

ID	Assumption	Meaning
A-MOD-08	Staking module summary interface	A modeled staking module summary supplies the deposited-validator count used by SRv3 exited-count validation. Node-operator internals and warning-event conditions are outside this lane unless a P0 property names them.
A-MOD-10	Staking module top-up allocation interface	A modeled `allocateDeposits` call returns the top-up allocation array used by SRv3 and does not mutate router accounting except through the explicit transition. Key ownership, queue-cursor behavior, and module internals remain outside this lane unless a P0 property names them.
A-MOD-11	Staking module total-stake interface	A modeled `getTotalModuleStake` call returns the total module stake used to compute WC02 allocation equivalents. Consensus-layer truthfulness and module-internal stake accounting remain outside this lane.
A-MOD-13	Reward-minted callback interface	A modeled `onRewardsMinted` callback is attempted only after the SRLib loop guards checked in P10. Callback effects, low-level revert bytes, event emission, and gas-estimation behavior remain outside this lane.
A-ALLOC-12	External allocation strategy	The external `MinFirstAllocationStrategy` and governance admissibility of target limits satisfy their documented contracts. This lane models the SRv3-owned current-allocation and capacity array construction before that strategy runs.

Explicit non-claims

Boundary	Treatment
BLS signatures, deposit roots, SSZ, Merkle proofs, EIP-4788	Named cryptographic and consensus-data boundaries, not part of the P0 accounting closure.
Exact packed-storage migration and proxy mechanics	Deferred to a deployment and migration lane; not used as proof evidence here.

Boundary	Treatment
Full governance role graph and parameter admissibility	Configuration is assumed to satisfy the stated range and role assumptions; unauthorized governance transitions are not modeled.
CL witness soundness and VaultHub	Not proved in this lane. If their values enter an accounting transition, they enter only through named, well-formed input assumptions; otherwise no theorem relies on them.
Gas, events, revert strings, and exact call-stack shape	Engineering and integration review surfaces, not conservation-law premises.

Allowed public wording

Precise claim

This report package demonstrates the executable local claim shape: the selected PR #1811 SRv3 accounting checks pass over a focused economic-state-machine model, under explicit assumptions for external contracts, accepted oracle inputs, cryptographic witnesses, governance configuration, and deployed-system refinement. Use broader theorem or deployed-system proof language publicly only after stronger artifacts are added and independently rebuilt.

4 Reproducibility and handoff

Reviewer command

A reviewer does not need to trust the PDF alone. The handoff includes the proof sources and commands that rebuild the selected executable target set and check that the PDF's assumptions match the machine-readable trust-boundary report.

The rebuild is the reviewer's receipt. If it succeeds, the reviewer learns that the target IDs in the PDF exist, the executable checks pass, and the listed assumptions match the machine-readable trust report. If it fails, the PDF claim is not ready to rely on.

```
git clone git@github.com:lfglabs-dev/lido-srv3-proof-closure.git
cd lido-srv3-proof-closure
make bootstrap
make test
make prove
make report
```

These commands check that the source fixtures are present, verify every matrix row has a matching Verity/Lean artifact in `proofs/`, emit `proofs/logs/proof-report.json`, and rebuild the PDF.

Minimum provenance fields

Field	Repository content
PR and source revision	PR #1811 URL, base branch, audited commit hash, and review timestamp. Report package target: `d088bbc2deac9913b68036d73d35c37aa6279b90`.
Proof toolchain	Lean/Lake project importing pinned Verity reference `33722270d996c7a3a520a71ecee42d7d232da100`; proof command `lake build LidoSRv3`.
Target namespace	Selected proof target files in `proofs/`, target names in `verity/targets/srv3-proof-targets.json`, and the command that fails on a failed model check.
Trust allowlist	Machine-readable assumption IDs, out-of-model boundaries, and target-to-assumption mapping in `verity/targets/trust-boundary.json`.
Artifact integrity	Generated PDF in `dist/lido-srv3-formal-methods-report.pdf` and proof output in `proofs/logs/proof-report.json`.

Source anchors

The matrix uses broad PR #1811 surfaces to stay readable. The exact source map is `verity/targets/source-map.json`. Anchors for the target source revision include:

- Buffer and reserve accounting: `Lido.sol::_getBufferedEtherAllocation`, `getDepositableEther`, and `withdrawDepositableEther`.
- Deposit allocation: `StakingRouter.sol::deposit` and `SRLib.sol::_getDepositAllocations`.

- Reports and rewards: `StakingRouter.sol::reportValidatorBalancesByStakingModule`, `getStakingRewardsDistribution`, and `SRLib.sol::_reportRewardsMinted`.
- Oracle validation: `AccountingOracle.sol::_checkStakingRouterModuleBalances`.

Handoff contents

- **PDF:** `dist/lido-srv3-formal-methods-report.pdf`. This report package for Lido reviewers.
- **Lockfile:** `proofs/LOCKFILE.md`. Pinned Lido and Verity references.
- **Trust report:** `verity/targets/trust-boundary.json`. Machine-readable assumption and boundary register.
- **Proof targets:** `proofs/`. One target file for each selected P0 property.
- **Model:** `LidoSRv3/Model.lean`. SRv3 economic state machine.
- **Theorems:** `LidoSRv3/SpecProofs.lean`. Checked P0 proof targets.
- **Reference tests:** `tests/solidity-reference/`. Copied Lido references used as source-facing fixtures.

Integrity checks

Check	Expected result
`make test`	Source-facing Solidity fixtures are present and no legacy proof artifacts remain.
`make prove`	Verity/Lean theorem checking passes SRV3-P1 through SRV3-P6 and writes `proofs/logs/proof-report.json`.
`make report`	Rebuilds `dist/lido-srv3-formal-methods-report.pdf`.
`proofs/LOCKFILE.md`	Contains Lido PR #1811 commit `d088bbc2deac9913b68036d73d35c37aa6279b90` and Verity commit `33722270d996c7a3a520a71ecee42d7d232da100`.

A Appendix A: Reader glossary

Router The SRv3 component that allocates deposit work across staking modules and records aggregate module accounting.

Module A staking-provider lane managed by the router.

Accepted report Oracle balance data that has passed the modeled SRv3 validation checks and may update accounting state.

Withdrawal reserve ETH kept aside for withdrawals, not available for new deposits.

CL Consensus layer, the Ethereum system that records validator activity.

Lean A proof checker that verifies the theorem files under `LidoSRv3/`.

Verity The modeling framework pinned for this work through `lakefile.lean` and imported by the SRv3 Lean package.

P0 Highest-priority property class. Here it means an accounting claim important enough that the executable package should either check it or clearly say why it remains open.

Proof target A precise executable check tied to source anchors, assumptions, model files, and proof logs.

Assumption A named premise that the executable target may use but does not prove.

Trust boundary The edge of the proof: what the model proves, what it assumes, and what remains for separate review.

B Appendix B: Certora delta

This lane does not replace earlier Certora work. It shows how a PR #1811 proof-closure package continues from that work: moving from bounded or summarized checks toward explicit SRv3 accounting transitions, balance-based module reasoning, and named assumptions that a reviewer can audit.

Legacy coverage class	Disposition	Final treatment
V2 validator-count accounting	Continued	Reframed as module-balance conservation over accepted per-module balances.
V2 deposit availability bounds	Re-expressed	Expressed as reserve separation plus exact 32 ETH pull accounting.
V2/V3 reward distribution checks	Re-expressed	Expressed as accepted-balance reward computation, fee bounds, and recipient alignment.
Router status-gating assertions	Reused and extended	Applied to active, stopped, and deposit-paused gates in the modeled economic paths.
V3 bounded module-array specs	Mirrored	Verity/Lean theorems cover the focused economic model with named well-formedness assumptions.

C Appendix C: Expected axiom and assumption summary

Class	Expected final value	Interpretation
Lean axioms introduced by LFG	0 known	The package ships the `LidoSRv3` Lean namespace and does not add custom axioms.
Admitted theorem bodies	0	The checked theorem files contain no `sorry` or `admit`.
Named model assumptions	14	The report package names each out-of-model interface and domain premise in `verity/targets/trust-boundary.json`, covering external summaries, governance/module-management plumbing, beacon sinks, accepted oracle reports, module identity/order well-formedness, arithmetic domains, reward precision, Lido withdrawals, staking-module callbacks, and allocation strategy behavior.
Selected falsification/regression harness counterexamples	0	`make prove` checks the selected P0 theorem file. This is not a claim about unmodeled deployed-system behavior.

D Appendix D: Follow-on lanes

These lanes are recommended because they touch adjacent SRv3 risk surfaces, not because they are required to support the accounting claims in this report package. Recommended sequence after this P0 accounting closure:

1. ExitBus authorization, replay resistance, duplicate prevention, and triggerable-exit fee/refund exactness.
2. Consolidation fee conservation, source and target binding, and source-ownership caveats.
3. Top-up consensus-layer proof binding beyond allocation accounting.
4. Oracle sanity limits, drain guards, and second-opinion oracle composition.
5. Deployment governance, role graph, and exact migration refinement.
6. VaultHub F-01/F-02/F-03, if still relevant to the active review path.